Sprint review and retrospective notes

Sprint 1: User authentication

Sprint Review Notes

Goal:

Develop user registration and login features.

Completed Deliverables:

1. Validate email and password.
2. Implemented error message handling for incorrect credentials.

Sprint Retrospective Notes

What Went Well:

We successfully implemented a basic registration system in C++ with input validation for email and password.

SonarCloud helped us identify important code quality issues early, such as inefficient parameter passing and unsafe header practices.

The team improved code structure by separating logic into multiple files (main.cpp, user.h, user.cpp), which made the project more organized and modular.

We applied best practices like using references to improve performance and reduce unnecessary copying.

What Could Be Improved:

We need to pay more attention to clean code principles from the beginning instead of fixing issues later.

Testing is still very basic; the program does not handle edge cases (e.g., complex email validation or secure password handling).

The system currently stores data only in memory, so adding file or database storage would improve functionality.

Team awareness of C++ best practices (like parameter passing and header file design) can be improved to avoid similar issues in future sprints.

Action Items for next sprint:

- Develop functionality to display the full list of courses in a clear and organized format.
- Create a search input field to allow users to enter keywords.
- Add filtering functionality to refine search results and display only matching courses.
- Ensure code follows clean code practices (e.g., proper structure, avoiding code smells, efficient parameter passing).
- Test the system with different inputs to ensure correct and reliable search results.

Sprint 2: Course Browsing

Sprint Review Notes

Goal:

Enable students to browse and search available courses.

Completed Deliverables:

1. displays all available courses.
2. Implemented the search input field and search functionality.
3. Filtered and displayed search results based on student keyword input.

Sprint Retrospective Notes

What Went Well:

The code had too many loops inside loops. We fixed this by moving the logic into separate functions,Some loops were written the long way. We replaced them with a shorter, cleaner way using **any_of (any_of is a ready-made tool in C++ that checks if at least one item in a list matches what you are looking for)**.

What Could Be Improved:

Right now passwords are stored as plain text inside the program. This means if someone looked at the data they could see every password directly. In a real system this is very dangerous. passwords should be scrambled using a technique called hashing before they are saved, so even the program itself cannot read them back.

Action Items for next sprint:

- If the code has too many loops inside each other, move them into their own functions early, do not wait until the end.
- When searching through a list, use any_of instead of writing a long loop.

Sprint 3: Course Registration

Sprint Review Notes

Goal:

Enable full course enrollment management for students.

Completed Deliverables:

1. Implemented course registration functionality.
2. Implemented course drop functionality that removes the course from the student's schedule.
3. Developed the student schedule page showing all currently enrolled courses.

Sprint Retrospective Notes

What Went Well:

During this sprint, the integration between GitHub and SonarQube was handled effectively. The team followed a structured workflow by creating feature branches, committing changes incrementally, and using pull requests for code integration.

SonarQube analysis was successfully used to identify code smells and maintainability issues early. Several issues, including variable scope improvements using C++17, were resolved efficiently. This led to cleaner and more readable code.

What Could Be Improved:

Some issues were only addressed after the pull request was created, which led to multiple iterations of fixes. This indicates that earlier local review or pre-checking against SonarQube rules could reduce rework.

Minor code smells accumulated during development instead of being fixed immediately, which slightly slowed down the final cleanup phase.

Action Items for Next Sprint:

- Follow coding best practices (such as proper variable scoping) during initial implementation

Maintain smaller, more frequent commits to simplify tracking and debugging